KASPARRO

AI Representation Optimizer for Shopify

Technical Document | Kasparro Agentic Commerce Hackathon | April 2026

Prepared by Aswin Kumar B S and Naveen D

1. System Architecture

Component Overview

Layer	Technology	Role
Frontend	React 18, Vite, TanStack Query, shadcn/ui	SPA on Vercel CDN; session-cookie auth; no server-side rendering
API Server	Node.js 20, Express, TypeScript, Drizzle ORM	REST API on Railway; all business logic, AI calls, Shopify writes
Database	Neon PostgreSQL, connect-pg-simple	16 tables; server-side sessions; append-only score snapshots
Primary AI	Google Gemini 2.0 Flash	Scoring quality assessment, fix generation, all feature modules
AI Fallbacks	OpenRouter Gemma 4 31B, Groq LLaMA 3.3 70B	Cascade fallback if Gemini fails; provider stats tracked in memory
GEO Integrations	Tavily, SerpAPI, AWS Bedrock Nova, OpenAI GPT-4o-mini	Multi-platform citation scanning; each tests a different AI surface
Shopify	Admin REST API, Webhooks (HMAC-signed)	Product sync, fix deployment, real-time catalog updates

Data Flow — Four Critical Paths

Flow	Step-by-Step Path
Analysis run	Browser POST /analysis -> jobs row created (idempotency key) -> AnalysisQueue (in-process, maxConcurrent=1) -> 5 products scored in parallel per batch -> each product: ~40 rules in <5ms + Gemini quality call -> 60/40 blend -> gaps upserted (deterministic IDs) -> fixes pre-generated -> store-level aggregate computed -> score snapshot inserted -> job marked done
Fix apply	Browser POST /fixes/:id/apply -> optional editedContent overrides generated content -> Shopify Admin REST PATCH product -> read back updated field from Shopify -> if read-back matches: update DB score, mark gap fixed, update fix status -> if read-back fails: return error, no score change
Webhook sync	Shopify POST -> verifyWebhookHmac (timingSafeEqual, raw body from express.json verify callback) -> ACK 200 immediately -> setImmediate() async processing -> update product fields in DB -> null aiQaCachedAt to force regeneration -> dedup check (60s in-memory Map) -> insert activity record
GEO scan	Browser POST /geo/scan -> derive queries from store's actual product types and price range -> 5 platform API calls in parallel (Tavily/SerpAPI/Bedrock/OpenAI/Gemini) ->

Flow	Step-by-Step Path
	each returns cited boolean + evidence -> persist as visibility_check records -> return aggregate with per-platform score arcs

2. What AI Does vs. What Deterministic Code Does — and Why

This is the most important architectural decision in Kasparro. We drew a deliberate line between what AI handles and what rule-based code handles. Getting this wrong in either direction has real costs.

The Line We Drew

Task	Handled By	Why This Side of the Line
Checking title word count	Deterministic rule	Objective, countable, no interpretation needed. AI adds latency with no accuracy gain.
Checking tag count >= 5	Deterministic rule	Binary check. Rule fires in microseconds. AI cannot do this faster or better.
Detecting passive voice	Deterministic rule	Regex + NLP pattern matching. Deterministic, auditable, fast.
Detecting JSON-LD schema presence	Deterministic rule	Checks for <script type='application/ld+json'> in product HTML. Either it is there or it is not.
Assessing whether content reads naturally	Gemini AI	Requires understanding — a description can pass all rules and still be stilted, repetitive, or unnatural. Only AI catches this.
Estimating AI citability of a description	Gemini AI	Requires simulating an AI agent's judgment. Cannot be reduced to rules without significant false positive/negative rates.
Generating improved product descriptions	Gemini AI	Creative synthesis. Rule-based templates produce generic output; AI preserves brand voice and context.
Analysing cross-product brand consistency	Gemini AI (batch)	Requires reading multiple products together and reasoning about coherence. No deterministic equivalent.
Categorising product type/vertical	Gemini AI	Product taxonomy mapping from free-text requires semantic understanding, not keyword matching.

Task	Handled By	Why This Side of the Line
Identifying FAQ gaps from policy text	Gemini AI	Reads policy page HTML and extracts unanswered topics. Requires reading comprehension.

Why the 60/40 Blend Specifically

The final dimension score is 60% rule engine + 40% Gemini. This ratio was reached empirically:

- Below 50% rules: score variance between runs (from Gemini non-determinism) was high enough to make trend charts unreliable — a product could 'improve' 8 points without any change to content.
- Above 70% rules: the AI contribution was too small to differentiate between technically-valid-but-weak content and genuinely strong content. Two products with identical rule scores but very different quality would receive the same final score.
- At 60/40: trends are stable (rule anchor), quality sensitivity is preserved (AI contribution), and the score is explainable — merchants can see which rules fired and understand why the AI portion moved the score.

Schema Resilience — When AI Returns Garbage

Every Gemini call in Kasparro uses structured output mode with an explicit JSON schema. When Gemini returns a malformed response or a response that fails schema validation, the system does not crash — it applies field-level defaults and completes the analysis with the rule-based portion only. Specifically:

- If the AI quality score is missing or out of range: the dimension score uses the rule score at full weight (effectively 100% rule, 0% AI for that dimension). The merchant sees the score with a 'rule-only' indicator.
- If the AI-generated fix content is empty or too short: the fix is stored as pending with a 'generation failed' status. The gap still shows; the merchant can manually edit and apply.
- If the AI brand voice assessment fails: the consistency dimension score falls back to the rule-only consistency checks (pricing signal integrity, policy statement checks).

3. Failure Handling — What Happens When Things Break

Shopify API Failures

Failure Mode	What Kasparro Does
Shopify API is down at analysis time	Analysis still runs against the product data already in Neon DB from the last successful sync. The merchant gets a score based on cached data with a 'Using cached catalog — sync Shopify to update' notice. No blank screens, no 500 errors.
Shopify API returns 429 (rate limited) on fix apply	The fix apply endpoint catches the 429, returns a structured error: { status: 'rate_limited', message: 'Shopify is rate limited. Wait 30 seconds and retry.' } The fix is not marked as applied. The gap remains open. The merchant retries manually.
Shopify API returns 200 but content did not update (silent truncation)	The read-back verification catches this. The API reads the product field back after writing. If it does not match the sent value, the fix is flagged as 'Sync uncertain — verify in Shopify admin' and the score is not updated. Without read-back, we would silently credit improvements that did not land.
Shopify webhook delivery fails (Shopify retries)	Kasparro ACKs 200 immediately — Shopify only retries if we do not respond within 5 seconds. Processing happens in setImmediate() after the ACK. If processing fails, the webhook is not re-delivered by Shopify, but the product data remains in DB from the last sync. A 60-second in-memory dedup Map prevents duplicate activity entries when Shopify delivers the same webhook twice.
Shopify OAuth token is revoked by merchant	The next API call (sync, fix apply) returns 401. The store is flagged as 'disconnected'. The merchant sees a reconnect prompt. Existing score data and history are preserved.

LLM / AI Provider Failures

Failure Mode	What Kasparro Does
Gemini returns HTTP error or timeout	Provider cascade activates: retry once on Gemini, then fall back to OpenRouter Gemma 4 31B (free tier), then Groq LLaMA 3.3 70B. Each provider uses the same prompt and schema. The _fallbackStats object tracks which provider served each request — exposed via /api/ai-fallback-stats for monitoring.
All three providers fail	Analysis completes using deterministic rules only (60% weight becomes 100%). Gaps are still generated from rules. Fixes are not generated — shown as 'AI unavailable, rule-based analysis only'. The merchant gets partial value; the system does not error out.
LLM returns malformed JSON	The AI client wrapper catches JSON.parse errors and schema validation failures. Field-level defaults are applied. The analysis continues. Gemini's structured output mode (response_mime_type: 'application/json') reduces this failure mode to near-zero in practice.

Failure Mode	What Kasparro Does
LLM returns plausible-looking but wrong content (hallucination)	For scoring: rule engine provides the anchor (60%) so a hallucinated AI score cannot move the result by more than 40%. For fix generation: content is shown to the merchant before applying — they are the last line of defence against hallucinated content. We do not auto-apply without merchant review.
GEO scan AI platform fails (one of five)	That platform's result is marked as 'unavailable' in the scan report. The overall GEO score is computed from the remaining platforms. A partial scan is more useful than a failed scan.

Unexpected User Input and Edge Cases

Failure Mode	What Kasparro Does
Merchant submits request for a store they do not own	Every route calls getOwnedStore(storeId, userId) which joins stores on userId. A mismatched userId returns null and the route returns 403 Forbidden. This prevents IDOR (Insecure Direct Object Reference) — a merchant cannot read, score, or modify another merchant's store data.
Analysis request submitted twice before first completes	The jobs table has an idempotency key. The second POST with the same key returns the existing job record immediately (200 with jobId) rather than starting a second analysis. The AnalysisQueue (maxConcurrent=1) also prevents concurrent executions even without the idempotency check.
Product with empty description or null fields	Rules that check description content fire against an empty string — they produce high-severity gaps ('Description is missing') with maximum impact score. Null fields are normalised to empty strings before rule execution. No null pointer exceptions.
Competitor domain returns no products (private catalog)	The competitor analysis returns an empty result set with a message: 'No products found on this domain — catalog may be private or domain may not be a Shopify store.' No crash, no partial score shown as real.
Webhook arrives with unrecognised store domain	The webhook handler looks up the store by normalised domain. If not found, it returns early (no-op). No error logged — unrecognised domains are expected when Kasparro is uninstalled but Shopify keeps delivering webhooks for a short window.
Session expires mid-flow	Protected routes return 401 { code: 'UNAUTHENTICATED' }. The frontend intercepts 401 responses globally and redirects to the login page. In-flight form data is preserved in component state so the merchant can log back in and continue.

4. Key Implementation Decisions

Decision	We Chose	Because
Decision	We Chose	Reasoning
Access token storage	AES-256-GCM in application layer	Avoids vendor lock to DB-level encryption; auth tag detects tampering; resolveAccessToken() handles legacy plaintext transparently without a migration
Gap IDs	Deterministic: gap_{storeId[-8]}_{productId[-8]}_{ruleId}	Enables INSERT...ON CONFLICT DO UPDATE — isFixed state survives re-runs without needing to track which gaps are new vs. existing
Cache invalidation	SHA-256 catalog hash + 24h TTL	Expensive AI feature calls only re-run when products actually changed; hash makes TTL irrelevant for merchants who edit products frequently
Webhook processing	ACK 200 + setImmediate()	Shopify retries if no ACK within 5s; decoupling processing from ACK means slow DB writes never cause retries
Fix generation timing	Pre-generate during analysis, not on-demand	3-8s AI latency at click time would destroy the one-click UX; storage cost of pre-generating is negligible vs. UX gain
Analysis concurrency	In-process queue, maxConcurrent=1	Protects Gemini rate limits and prevents DB write contention on shared Railway instance; acceptable for current scale

5. Known Limitations and What We Would Fix with More Time

Limitation	What We Would Do with More Time
In-memory generation locks and dedup Map are lost on server restart	Replace with Redis SETNX for distributed locks and a webhook_idempotency table for durable dedup. Currently a server restart during an active analysis leaves the job in 'running' state permanently — requires manual DB cleanup.
Single Railway instance, no horizontal scaling	Migrate analysis queue to Bull + Redis; make the API server stateless; add Railway horizontal scaling. The current architecture is correct for one server but cannot scale past ~50 concurrent analysis requests.
No pagination on gaps and fixes endpoints	Critical at scale — a store with 500 products can have 5,000+ gaps. Currently all returned in one response. Add cursor-based pagination with a page_size limit.
GEO scan queries are constructed from catalog metadata only	The queries should also incorporate the merchant's desired positioning field and actual buyer language from reviews. Currently a merchant selling 'ergonomic office chairs' might get queries for 'chair' if the product type field is sparse.
The 60/40 blend ratio was set empirically on a small test set	Run a structured evaluation on a labelled dataset of product descriptions (human-rated AI citability) to calibrate per-vertical blend ratios. Electronics content may need a higher rule weight; lifestyle content may need a higher AI weight.
Schema injection into Shopify theme is manual	Build a theme extension that auto-injects the generated JSON-LD without requiring the merchant to copy-paste into their theme code. Requires Shopify Theme App Extension API which we scoped out of the hackathon build.